LZ 压缩编解码器

姓 名：欧 思 逸

学 号：24300720245

数据结构与算法设计
（秋季学期，2025）

复旦大学
智能机器人与先进制造创新学院

目 录

1 项目概述与分析 **2**

1.1 项目概述 2

1.2 LZ 算法 2

2 朴素匹配 LZ 编解码器 **2**

2.1 算法设计与实现 2

2.1.1 数据结构设计 2

2.1.2 算法思路 3

2.1.3 特殊情况处理 3

2.2 朴素匹配压缩结果与分析 4

3 引入哈希字典加速 LZ 编码 **4**

3.1 Knuth 乘法哈希 4

3.2 Hash-Chain 5

3.2.1 数据结构设计 6

3.2.2 算法思路 6

3.3 Last-Pos Hash 7

3.3.1 优化路径：从朴素 Last-Pos 到双路哈希 8

3.3.2 数据结构设计 8

3.3.3 算法思路 9

3.4 样例测试与对比 9

3.4.1 测试环境 9

3.4.2 测试指标说明 10

3.4.3 候选位置数量限制 (针对 Hash-Chain) 10

3.4.4 哈希表容量限制 (针对 Last-Pos) 11

3.4.5 综合对比结论 12

4 Deflate 压缩算法优化 **12**

4.1 Defalte 算法简述 12

4.2 代码优化 13

4.3 优化前后压缩性能对比 14

4.3.1 Hash-Chain 算法分析 14

4.3.2 Last-Pos 算法分析 15

5 总结 **15**

5.1 核心工作总结 15

5.2 时空权衡与适用场景分析 15

1 项目概述与分析

1.1 项目概述

本项目实现了基于 LZ77 算法的文件压缩与解压工具，并在此基础上进行了加速和优化。项目主要实现路径包括：

1. 朴素匹配压缩
2. 哈希字典加速匹配
3. 利用 Huffman 编码实现 deflate 压缩

此外，项目中历史窗口大小固定为 32KB，最小匹配长度 $MIN_MATCH = 3$。

1.2 LZ 算法

LZ（Lempel-Ziv）算法的核心思想是在数据流中查找重复片段，并用 $(distance, token)$ 的匹配段替代重复内容；而对于无法压缩的字节，则使用字面量 (literal) 直接写出。

压缩流程如下：

1. 读取当前位置，在历史窗口中查找最长历史匹配
2. 若匹配长度 $\leq MIN_MATCH$，则输出 match，否则输出字面量
3. 更新位置

Token 输出格式选择：

- 每个 token 包含一个字节：`TOKEN_LITERAL=0` 表示字面量;`TOKEN_MATCH=1` 表示匹配段。
- `Match token` 包含两字节 distance 和 1 字节 length;`Literal token` 则直接储存字节值。

2 朴素匹配 LZ 编解码器

2.1 算法设计与实现

2.1.1 数据结构设计

1. **历史窗口表示**：
   - 使用数组 `buffer[]` 存储整个待压缩数据流。
   - 逻辑上的历史窗口大小固定为 32 KB (`WINDOW_SIZE = 32768`)，实际窗口通过限制匹配起始位置的索引范围实现。
   - 当前处理位置记为 `pos`，历史窗口范围为 $[\max(0, pos - \texttt{WINDOW_SIZE}), pos)$。
2. **Token 表示**：
   - 输出 token 通过单字节头部区分类型，高位（bit7）表示 token 类型：

- `bit7 = 0`：literal token（长字面量）。当当前位置没有可压缩匹配时，编码器连续收集多个无法匹配的字节，合并为一个 literal token。格式为：

$$[0|length(7 \text{ bits})]\ [byte_1]\ [byte_2]\ \ldots\ [byte_{length}]$$

  低 7 位表示字面量长度，最大 127。
- `bit7 = 1`：match token。当找到长度不小于 `MIN_MATCH` 的匹配时输出：

$$[1|length(7 \text{ bits})]\ [distance_high]\ [distance_low]$$

  其中低 7 位表示匹配长度，两个字节表示回溯距离。

3. **匹配结果表示**：
   - 在匹配搜索阶段，使用 `length` 和 `distance` 两个变量记录当前最优匹配结果。
   - 该二元组在逻辑上等价于一个匹配结果结构体，用于指导编码器输出对应的 match token。

2.1.2 算法思路

1. **匹配搜索（朴素匹配）**：
   - 对当前位置 `pos`，遍历历史窗口内的每一个候选起始位置 `i`。
   - 使用**逐字节比较**的方法，计算从 `i` 到 `pos` 的最长匹配长度 `length`。
   - 遍历完成后，选择匹配长度最长的位置作为当前的匹配结果。
2. **输出逻辑**：
   - 如果最长匹配长度 `length` $\geq$ `MIN_MATCH`，输出 match token：
     - 写入 token 类型标识 `TOKEN_MATCH`。
     - 写入距离 `distance = pos - i` 和匹配长度 `length`。
     - 将 `pos` 移动 `length` 个字节。
   - 否则输出 literal token：
     - 从当前位置开始连续收集无法匹配的字节，直到遇到可匹配段或达到最大 literal 长度 127。
     - 写入 token 类型标识（高位 0）和字面量长度、字节内容。
     - 将 `pos` 移动 `lit_len` 个字节。
3. **滑动窗口更新**：
   - 朴素匹配不使用额外数据结构维护历史窗口，只依赖数组索引和 `pos`。
   - 每次匹配或输出 literal 后，窗口自动向前滑动。

2.1.3 特殊情况处理

1. 当输入文件长度小于 `MIN_MATCH` 时，仅输出 literal，避免数组越界。
2. 匹配长度最大为 127，确保长度可用 7 位存储。
3. **连续多个 literal 会合并为单个 literal token**，减少格式开销。
4. Token 输出格式通过最高位区分 literal 与 match，保证解码器正确解析。

2.2 朴素匹配压缩结果与分析

本次实验选择了五个具有代表性的短文本文件作为测试对象，分别记录了原文件大小、压缩后文件大小、压缩率、平均压缩时间以及压缩带宽。实验多次运行取平均值，结果如表所示。

测试文本	原文件大小	压缩后大小	压缩率	压缩带宽
short_code.c	243B	190B	0.7819	0.04MB/s
config.json	317B	220B	0.694	0.08MB/s
random_text.txt	134B	136B	1.0149	0.03MB/s
html_snippet.html	386B	245B	0.6347	0.09MB/s
short_poem.txt	369B	358B	0.9702	0.08MB/s

可以看出，朴素匹配实现的 LZ 编码器在不同类型文本上的压缩效果存在明显差异：

1. **压缩率分析**：
   - 对于高冗余或结构重复较明显的文本（如 `html_snippet.html` 和 `config.json`），压缩率较低，说明重复片段较多，LZ 算法能有效利用历史窗口减少冗余。
   - 对于短源代码或随机文本（如 `short_code.c`、`random_text.txt`、`short_poem.txt`），压缩率接近或略高于 1，尤其是随机文本，压缩后文件甚至略大于原文件，这符合 LZ 算法的特点：当重复片段少或文件较短时，头部 token 开销可能导致“压缩膨胀”。
2. **压缩带宽分析**：

   压缩带宽受文本大小影响显著，小文件的带宽相对较低，而较大或连续内容的文件带宽较高，这说明 I/O 和循环匹配开销在小文件中占比更大。

综上，朴素匹配方法虽然实现简单，但在小文件或随机数据上效率有限，容易出现压缩膨胀。而对于有明显重复结构的文本，压缩效果明显；这也验证了 LZ 算法“利用历史窗口查找最长匹配”的核心思路。因此，若希望在所有文件上都有稳定压缩率和带宽，则考虑引入哈希字典等优化方法以减少匹配搜索时间。

3 引入哈希字典加速 LZ 编码

为了改进暴力匹配的性能，可以使用哈希字典加速匹配过程，不同哈希字典的结构对于压缩率和压缩带宽都会带来影响。

哈希字典可以加速 LZ 算法中的前缀匹配过程：维护历史窗口中，在处理 pos 处字符时，取接下来三个或者多个字符计算 key，然后去字典中取到候选位置，然后再一一尝试找到最优的匹配。

为了达到较好的压缩率，我们尝试了多种哈希函数以减小哈希冲突，最终选择使用 **Knuth 乘法哈希**。

3.1 Knuth 乘法哈希

Knuth 乘法哈希（Multiplicative Hash）是一种经典的哈希函数设计方法，其核心思想是：

- 对于给定的整数键 k，先将其乘以一个常数 A，通常选取 A 为 $0 < A < 1$ 的实数或 2^w 下的奇数常量；

- 然后取乘积的高位（或低位）作为哈希值，再对哈希表大小 m 取模：

$$h(k) = \lfloor m \cdot ((k \cdot A) \bmod 1) \rfloor$$

其中 $(k \cdot A) \bmod 1$ 表示取 $k \cdot A$ 的小数部分。

项目中使用了如下哈希函数：

```
    static inline unsigned int hash_func(unsigned char *p) {
    uint32_t val = ((uint32_t)p[0])
                 | ((uint32_t)p[1] << 8)
                 | ((uint32_t)p[2] << 16);
    return (val * 0x1e35a7bd) >> (32 - HASH_BITS);
}
```

其原理可以分解如下：

1. **读取输入键：** 将输入数组的前 3 个字节合并成一个 24 位整数 `val`：

$$\texttt{val} = p[0] + (p[1] \ll 8) + (p[2] \ll 16)$$

这样可以将连续的字节序列映射为整数键 k。

2. **乘以大素数：** `val * 0x1e35a7bd` 对应 Knuth 乘法哈希中的乘法步骤。这里的常数 $0x1e35a7bd$ 是一个大素数，用于打散原始整数的比特分布，保证哈希值均匀。相当于 Knuth 公式中的：

$$h(k) = \lfloor m \cdot ((k \cdot A) \bmod 1) \rfloor$$

3. **取高位作为哈希值：** `>> (32 - HASH_BITS)` 操作相当于取乘积的高位。在 Knuth 乘法哈希理论中，取乘积的小数部分或高位可以获得更均匀的分布。而最终输出值在 $[0, 2^{HASH_BITS} - 1]$ 范围内，用作哈希表索引。

总结 本项目中所选择使用的哈希函数体现了 Knuth 乘法哈希的特点：

- 通过大素数乘法实现比特扩散，使输入键的相似性不会导致哈希冲突。
- 取高位代替取模操作，提高计算效率。
- 输出范围通过 `HASH_BITS` 控制，方便映射到固定大小的哈希表。

3.2 Hash-Chain

哈希链（Hash-Chain）LZ 编码器在朴素匹配的基础上，引入哈希表与前驱链表结构，以加速历史匹配位置的查找。其核心思想是在保证窗口约束的前提下，仅在具有相同哈希值的历史位置中进行有限深度的匹配搜索，从而在压缩率与时间复杂度之间取得平衡。

其主要设计思想如下：

1. **利用哈希表快速定位候选匹配起始位置**

对当前位置的连续 3 个字节计算哈希值，并将该位置插入对应哈希桶的链表头部。匹配查找时，仅需遍历同一哈希桶中的前驱链表节点，避免对整个历史窗口进行线性扫描。

2. **限制搜索深度与窗口范围**

   通过限制哈希链的最大遍历深度（MAX_CHAIN），并在匹配过程中检查回溯距离是否超过窗口大小（WINDOW_SIZE），有效避免最坏情况下的过长搜索，保证算法的稳定运行时间。

3.2.1 数据结构设计

1. **哈希链表**：为了兼顾效率和内存使用，代码采用数组模拟哈希链表的方式存储历史位置，具体设计如下：

   - **哈希表头数组**：head[HASH_SIZE]，存储每个哈希值对应的链表最新位置（头指针），初始值为-1。
   - **前驱位置数组**：prev[] 用于记录同一哈希桶中每个位置的前一个历史位置，从而构成单向前驱链表。
   - **哈希函数**：基于当前位置的 3 个字节构造 24 位整数，并采用 **Knuth 乘法哈希方法**，将结果映射到 HASH_SIZE 范围内：

```c
static inline unsigned int hash_func(unsigned char *p) {
    // 读取3个字节作为整数
    uint32_t val = ((uint32_t)p[0]) | ((uint32_t)p[1] << 8) | ((uint32_t)p[2] << 16);
    // 0x1e35a7bd 是一个大素数乘数
    return (val * 0x1e35a7bd) >> (32 - HASH_BITS);
}
```

   该哈希函数具有较好的分布均匀性，可有效减少哈希冲突。

2. **Token 表示（压缩数据的输出格式）**：采用 7 位长度 + 标志位的协议格式，分为字面量（Literal）和匹配段（Match）两种 Token：

   - **字面量 Token**：1 字节头部（最高位为 0，低 7 位表示字面量长度），后跟对应长度的原始字节序列。
   - **匹配 Token**：1 字节头部（最高位为 1，低 7 位表示匹配长度），后跟 2 字节的回溯距离（高字节在前，低字节在后）。

3. **输出缓冲区管理**：为减少频繁的文件 I/O 操作，设计 OutBuffer 结构体作为输出缓冲区，在缓冲区写满时再统一写入文件，从而提升整体压缩效率。

4. **匹配结果存储**：匹配结果通过 best_len 和 best_dist 两个变量记录当前找到的最佳匹配长度和回溯距离。

3.2.2 算法思路

1. 预处理

   - 将整个输入文件一次性读取至内存缓冲区 buffer。
   - 根据文件大小动态分配 prev 数组，用于构建哈希前驱链表。
   - 将 head 数组初始化为 -1，表示初始状态下所有哈希桶为空。
   - 初始化输出缓冲区 OutBuffer，用于缓存压缩输出数据。

2. **哈希链表插入**：在遍历输入缓冲区的过程中，对每个位置 `pos` 计算哈希值，并将当前位置插入对应哈希桶的链表头部，同时维护前驱指针关系。

3. **匹配搜索**：

   - 从当前哈希桶中最接近的历史位置开始，沿前驱链表向后遍历。
   - 遍历深度不超过 `MAX_CHAIN`，并在回溯距离超过 `WINDOW_SIZE` 时提前终止搜索。
   - 采用快速字节比较策略：仅当潜在匹配有可能超过当前最优匹配长度时，才进行完整逐字节比较。
   - 在匹配长度达到 `MAX_MATCH`（127）时立即终止搜索。

4. **匹配结果处理与输出**：

   - 若找到长度不小于 `MIN_MATCH` 的匹配，先输出此前累积的字面量块，再输出匹配 Token。
   - 为提高后续匹配命中率，对匹配段中被跳过的位置补充执行哈希链表插入。
   - 若未找到有效匹配，则当前位置暂不输出，继续向前移动以累积字面量。

5. **结束处理与异常情况**

   - 遍历结束后，将剩余未输出的字面量统一写出。
   - 对空文件直接返回，不进行压缩处理。
   - 对内存分配失败等异常情况进行检测，确保程序安全退出。

3.3 Last-Pos Hash

Last-Pos 编码器严格遵循“每个哈希桶仅保存最近一次出现位置”的定义，从而将匹配查找的时间复杂度降低为常数级 $O(1)$。这种设计显著减少了内存访问与分支判断开销，有利于提升整体压缩带宽。

然而，这种方式的缺点也十分显著，匹配丢失的问题十分显著，以至于压缩率大大下降，因此我们选择采用**双路哈希（Dual-Hash）**与**大容量哈希表**相结合的策略，在不引入链表的前提下提升匹配命中率，从而提高压缩率至目标值。

其核心思想如下：

1. **无链表的常数时间查找**

   对于任一输入位置，仅通过哈希索引直接访问对应桶中的最近历史位置，不再维护冲突链表。相比 Hash-Chain 需要遍历多个候选节点，Last-Pos 的匹配查找为严格的 $O(1)$ 操作。

2. **双路哈希机制（Dual-Hash）**

   为降低哈希冲突导致的有效匹配丢失，算法同时维护两套独立的哈希索引结构：

   - 基于 **3 字节前缀**的哈希表，用于捕获短匹配；
   - 基于 **4 字节前缀**的哈希表，用于捕获更长且冲突概率更低的匹配。

   在匹配阶段，算法同时查询两张哈希表，并在可行的情况下选择匹配长度更长的一项作为最终结果，从而在常数时间内获得“第二次查找机会”。

3. **大容量哈希表降低冲突概率**

   哈希索引位数设为 `HASH_BITS = 20`，即每张表包含 2^{20} 个桶，远大于 32KB 滑动窗口内可能出现的不同子串数量，利用**数据稀疏性**显著降低哈希冲突发生的概率。

3.3.1 优化路径：从朴素 Last-Pos 到双路哈希

在 Last-Pos 编码器的实现过程中，我们发现用单纯的 lastpos 难以满足所给样例测试对与压缩率的要求。因此针对哈希冲突导致的压缩率瓶颈，我们提出了双路哈希（Dual-Hash）策略。

1. **朴素 Last-Pos 的不足**

   最初我们实现了一个基于标准 16-bit 哈希表（64K 桶）的朴素 Last-Pos 版本。由于对 lastpos 定义的限制，该算法面临严重的**哈希冲突（Hash Collision）**问题。

   当两个不同的字符串（如"ABC" 和"XYZ"）计算出相同的哈希值时，新出现的"XYZ" 会直接覆盖桶中原有的"ABC" 的位置信息。由于没有链表保存历史记录，算法被迫 "遗忘" 了"ABC" 的存在。当"ABC" 再次出现时，算法无法回溯找到之前的匹配，只能将其作为字面量输出。测试数据显示，这种 "覆盖即遗忘" 的特性导致压缩率高达 80%，无法满足 <55% 的指标。

   这主要是因为 Hash-Chain 算法通过链表结构，在发生冲突时保留了历史记录，而 LastPos 算法相当于只有一次记忆。

   要解决此问题，必须降低冲突概率。虽然扩大哈希表（从 16-bit 扩至 20-bit）能利用数据的稀疏性减少冲突，但对于高频出现的短字符串（3 字节），单纯依赖一个哈希函数仍存在盲区。

2. **双路哈希（Dual-Hash）策略**

   基于上述分析，我们设计了双路哈希机制。除了维护基础的 3 字节哈希表（`lastpos3`）外，额外引入了基于 4 字节前缀的哈希表（`lastpos4`）。

   - **容错机制**：3 字节哈希的冲突概率相对较高，容易被覆盖；而 4 字节哈希的区分度更高，冲突概率极低。
   - **互补优势**：当 `lastpos3` 因冲突失效时，`lastpos4` 往往能充当 "备份记忆"，找回丢失的匹配。

   这种设计相当于在 $O(1)$ 的时间复杂度下，给了算法第二次查找机会。

3.3.2 数据结构设计

1. **双路 Last-Pos 哈希表**：

   - `lastpos3[HASH_SIZE]`：基于当前位置前 3 个字节计算哈希值，存储该哈希值最近一次出现的位置；
   - `lastpos4[HASH_SIZE]`：基于当前位置前 4 个字节计算哈希值，提供更高区分度的匹配索引。

2. **双路哈希函数**：两种哈希函数均采用 Knuth 乘法哈希，但使用不同的乘数以避免产生相同的冲突模式：

```c
// 基于前 3 字节
static inline uint32_t hash3(const unsigned char *p) {
uint32_t val = (p[0] << 16) | (p[1] << 8) | p[2];
return (val * 0x1E35A7BD) >> (32 - HASH_BITS);
}

// 基于前 4 字节
static inline uint32_t hash4(const unsigned char *p) {
uint32_t val = *(const uint32_t *)p;
```

```
return (val * 0x9E3779B9) >> (32 - HASH_BITS);
}
```

3. **Token 表示与输出缓冲**

   压缩数据的 Token 格式与 Hash-Chain 编码器保持一致，采用 7-bit 长度字段与 1-bit 类型标志位的组合；输出端通过缓冲区批量写入文件以减少 I/O 开销。

3.3.3 算法思路

1. **预处理**

   将输入文件整体读入内存缓冲区，并初始化 `lastpos3` 与 `lastpos4` 数组为 -1，表示尚无历史记录。

2. **双路匹配搜索**

   对每个位置 `pos`，执行以下常数时间匹配流程：

   - 查询 `lastpos3`，在窗口范围内验证基于 3 字节前缀的匹配长度；
   - 若剩余数据长度允许，则查询 `lastpos4`，验证基于 4 字节前缀的匹配；
   - 在两者中选择匹配长度更长的一项作为当前最佳匹配结果。

3. **匹配输出与全量更新**

   - 当匹配长度不小于 `MIN_MATCH` 时，先输出累积的字面量，再输出匹配 Token；
   - 随后对匹配段中被跳过的每一个位置执行哈希更新（Full Update），确保这些位置在未来仍可作为匹配起点；
   - 若当前位置未找到有效匹配，则单步前进并更新哈希表。

4. **覆盖式写入策略**

   每次哈希更新都会直接覆盖对应桶中的旧位置值，不保留历史记录，这是 Last-Pos 算法在空间与速度上的核心取舍。

3.4 样例测试与对比

3.4.1 测试环境

1. **CPU 型号**：Apple M4
2. **操作系统**: macOS 15.6.1
3. **编译器**：Apple clang version 17.0.0 (clang-1700.0.13.5)
4. **测试模式**：单线程
5. **正确性验证**: 使用 `verify_correctness` 工具对解压后的文件与源文件进行**逐字节核对（Byte-by-Byte Verification）**。测试要求解压文件与源文件必须实现 100% 的无损还原，任何字节层面的差异均被判定为未通过正确性验证。

3.4.2 测试指标说明

为了测试各类压缩算法的性能，设置如下指标。（所有指标的计算均在文件 `testtest_lz.c` 中完成）

1. **原始大小 (MB)**：待压缩文件的实际存储大小，单位为兆字节（MB）。
2. **压缩后大小 (MB)**：文件经过压缩算法处理后的大小，单位为兆字节（MB）。
3. **压缩率 (%)**：压缩算法的效率指标，用于衡量压缩后文件占原文件的比例，计算公式为：

$$\text{压缩率 (\%)} = \frac{\text{压缩后大小 (MB)}}{\text{原始大小 (MB)}} \times 100\%$$

   压缩率越低，说明压缩效果越好。

4. **平均耗时 (s)**：算法对指定文件进行压缩操作所需的平均时间，单位为秒。为了提高测试可靠性，通常会对同一文件**重复 3 次压缩**，并取平均值：

$$\text{平均耗时 (s)} = \frac{\sum_{i=1}^{3} t_i}{3}$$

   其中 t_i 为第 i 次压缩耗时。

5. **压缩带宽 (MB/s)**：衡量算法处理速度的指标，表示单位时间内可压缩的原始数据量，计算公式为：

$$\text{压缩带宽 (MB/s)} = \frac{\text{原始大小 (MB)}}{\text{平均耗时 (s)}}$$

   带宽越大，说明算法压缩速度越快。

6. **正确性验证**：用于检验压缩算法在压缩与解压缩后数据是否完整。若解压后的文件与原文件完全一致，则标记为 `PASSED`。

说明 本测试中，文件大小的计算基于程序读取的真实字节数，并采用二进制单位（1MB = 1024^2 Bytes）。而操作系统文件管理器中显示的文件大小采用十进制单位（1GB = 1000^3 Bytes）。因此，同一文件（如 enwik9）在程序中显示约为 953MB，而在系统中显示为 1GB，该差异来源于单位定义不同，并不影响本测试结果的正确性。

3.4.3 候选位置数量限制 (针对 Hash-Chain)

此处我们选择 MAX_CHAIN（最大哈希链搜索深度）作为控制变量。该变量参数控制回溯历史的深度，深度越大，找到更优匹配的概率越高，但计算量成倍增加。

通过调整 `MAX_CHAIN` 参数，控制在哈希链表中最多回溯多少个历史节点。

- `MAX_CHAIN = 4`：快速模式，仅看最近 4 个结果。
- `MAX_CHAIN = 32`：标准模式。
- `MAX_CHAIN = 64`：高压缩率模式，搜索更深。

配置	文件名	原始大小(MB)	压缩后大小(MB)	压缩率%	平均耗时(s)	压缩带宽(MB/s)	正确性
4	enwik8	95.37	54.95	57.62	0.6510	146.49	PASSED
4	enwik9	953.67	486.1	50.97	5.4442	175.17	PASSED
32	enwik8	95.37	49.14	51.53	1.2258	77.80	PASSED
32	enwik9	953.67	435.26	45.64	10.7171	88.99	PASSED
64	enwik8	95.37	48.48	50.84	1.5509	61.49	PASSED
64	enwik9	953.67	429.89	45.08	12.9930	73.40	PASSED

图 1: 不同候选位置限制下的 Hash-Chain 性能

分析:

- enwik8 压缩率：57.62% → 51.53% → 50.84%

enwik9 压缩率：50.97% → 45.65% → 45.08%

- enwik8 压缩带宽：146.49MB/s→ 77.80MB/s → 61.49MB/s

enwik9 压缩带宽：175.17MB/s→ 88.99MB/s → 73.40MB/s

随着搜索深度 `MAX_CHAIN` 的增加，算法能够找到更优的匹配，从而降低压缩率；但同时因为遍历链表的开销增大，压缩带宽呈现下降趋势。当深度为 32 时，达到了较好的平衡。

3.4.4 哈希表容量限制 (针对 Last-Pos)

由于 Last-Pos 定义限制了候选位置只能为 1，我们转而测试哈希表大小 `HASH_BITS` 对冲突率及压缩率的影响。

- `HASH_BITS = 16` (64K 桶)：标准大小。
- `HASH_BITS = 20` (1M 桶)：大容量。
- `HASH_BITS = 24` (16M 桶)：超大容量，模拟“无限记忆”。

配置	文件名	原始大小(MB)	压缩后大小(MB)	压缩率%	平均耗时(s)	压缩带宽(MB/s)	正确性
16	enwik8	95.37	55.40	58.09	0.6310	151.14	PASSED
16	enwik9	953.67	498.42	52.26	4.5627	209.01	PASSED
20	enwik8	95.37	54.93	57.60	0.7498	127.20	PASSED
20	enwik9	953.67	494.47	51.85	4.6463	205.26	PASSED
24	enwik8	95.37	54.91	57.58	0.8156	116.94	PASSED
24	enwik9	953.67	494.31	51.83	5.7364	166.25	PASSED

图 2: 不同哈希表大小下的 Last-Pos 性能

分析:

- enwik8 压缩率：58.09% → 57.60% → 57.58%

enwik9 压缩率：52.26% → 51.85% → 51.83%

- enwik8 压缩带宽：151.14MB/s→ 127.20MB/s → 116.94MB/s

enwik9 压缩带宽：209.01MB/s→ 205.26MB/s → 166.25MB/s

在标准 16-bit 哈希表下，Last-Pos 因严重的哈希冲突导致压缩率较差。随着哈希表容量扩大（20-bit 和 24-bit），冲突概率呈指数级下降，压缩率有所提升。值得注意的是，无论哈希表多大，由于查找操作始终是 $O(1)$，其压缩带宽始终维持在极高水平，远超 Hash-Chain 方法。

3.4.5 综合对比结论

在 enwik8 和 enwik9 测试中，对比两种算法的合适配置 (hash-chain 搜索深度 =32,last-pos 哈希表大小 =20)，如下表所示：

哈希字典选择	文件名	压缩率%	平均耗时(s)	压缩带宽(MB/s)
hash-chain	enwik8	51.53	1.2258	77.80
lastpos	enwik8	57.60	0.7498	127.20
hash-chain	enwik9	45.64	10.7171	88.99
lastpos	enwik9	51.85	4.6463	205.26

图 3: 两种哈希字典性能对比

- **压缩率**：Hash-Chain (Depth 32) 略优于 Last-Pos，因为它能解决哈希冲突，找回久远的历史匹配。
- **带宽**：Last-Pos 具有压倒性优势，通常比 Hash-Chain 快 2-3 倍。

本实验成功实现了 Hash-Chain 和 Last-Pos 两种哈希字典匹配方法。实验表明，虽然 Last-Pos 算法受到仅存最近位置的定义限制，但通过引入**双路哈希**和**大容量哈希表**等手段，可以有效克服其健忘缺陷，在保持高压缩带宽的同时，实现了与链表法相当的压缩率。

4 Deflate 压缩算法优化

4.1 Defalte 算法简述

Deflate 是一种经典的无损压缩算法，由 Phil Katz 提出，并被标准化于 RFC 1951 中。该算法采用 LZ77 字符串匹配与 Huffman 熵编码相结合的两阶段压缩框架，在保证压缩效率的同时兼顾了实现复杂度与解码速度。

在 Deflate 算法中，输入数据首先通过 LZ77 机制被表示为两类符号：

- **字面量（Literal）**，表示无法匹配的原始字节；
- **匹配对（Length, Distance）**，表示对历史窗口中已出现子串的引用。

随后，Deflate 并不直接以固定字节格式输出这些符号，而是进一步对字面量、长度码和距离码进行 Huffman 编码，利用不同符号出现频率的不均匀性，以更短的比特序列表示高频符号，从而显著降低整体输出比特数。

Deflate 标准支持动态 Huffman 树与静态 Huffman 树两种模式。其中，静态 Huffman 使用预定义、固定的编码表，避免了传输 Huffman 树结构的额外开销，在实现简洁性和压缩效果之间取得了良好的平衡。

4.2 代码优化

在前述 Hash-Chain / Last-Pos LZ77 编码器的基础上，我们进一步引入了 Deflate 算法中的静态 Huffman 编码机制，对压缩数据的输出格式进行优化。需要指出的是，此次优化代码并未完整复现 Deflate 标准的所有特性，而是有选择地引入其中对压缩率提升最为关键的部分。

在未引入 Deflate 优化前，编码器直接以字节为单位输出字面量和匹配对，其格式虽然简单，但未充分利用符号分布的不均匀性。

为此，本实现采用 **RFC 1951 中定义的 Deflate 静态 Huffman 编码表**，对输出符号进行比特级编码：

- 字面量字节（0–255）使用静态 Huffman 表编码；
- 匹配长度首先映射为 Deflate 规范中的长度码（257–285），并附加必要的额外比特；
- 匹配距离映射为距离码（0–29），并输出对应的固定 5 位 Huffman 码及额外比特。

```
// 长度编码映射表: base_length, extra_bits
static const struct { unsigned short base; unsigned char bits; } len_table[29] = {
    {3,0}, {4,0}, {5,0}, {6,0}, {7,0}, {8,0}, {9,0}, {10,0},
    {11,1}, {13,1}, {15,1}, {17,1},
    {19,2}, {23,2}, {27,2}, {31,2},
    {35,3}, {43,3}, {51,3}, {59,3},
    {67,4}, {83,4}, {99,4}, {115,4},
    {131,5}, {163,5}, {195,5}, {227,5},
    {258,0} };
```

```
// 距离编码映射表: base_dist, extra_bits
static const struct { unsigned short base; unsigned char bits; } dist_table[30] = {
    {1,0}, {2,0}, {3,0}, {4,0},
    {5,1}, {7,1}, {9,2}, {13,2},
    {17,3}, {25,3}, {33,4}, {49,4},
    {65,5}, {97,5}, {129,6}, {193,6},
    {257,7}, {385,7}, {513,8}, {769,8},
    {1025,9}, {1537,9}, {2049,10}, {3073,10},
    {4097,11}, {6145,11}, {8193,12}, {12289,12},
    {16385,13}, {24577,13}
};
```

这种**分段编码方式**能够以较少的 Huffman 码覆盖更大的取值范围，是 Deflate 在压缩效率上的重要设计之一。

此外，为支持 Huffman 编码的比特级输出，编码器实现了独立的**比特流缓冲机制**。在输出过程中，所有 Huffman 码均按照 Deflate 规范要求采用 **LSB-first（低位优先）**的方式写入比特流，并在必要时进行比特反转处理，以保证编码结果符合标准定义。

同时，为降低代码复杂度，我们并未使用动态 Huffman 编码，而是使用 Deflate 中的 **固定 Huffman 块**（BTYPE = 01）。相较于动态 Huffman 编码，固定 Huffman 模式避免了构建与传输 Huffman 树的额外开销。

4.3 优化前后压缩性能对比

我们仍然选择 enwik8、enwik9 作为两个测试文件，测试结果如下表。

fang哈希字典选择	是否引入Deflate	文件名	压缩率%	平均耗时(s)	压缩带宽(MB/s)	正确性验证
hash-chain	\	enwik8	51.53	1.2258	77.80	PASSED
hashchain	√	enwik8	45.12	1.5409	61.89	PASSED
hash-chain	\	enwik9	45.64	10.7171	88.99	PASSED
hashchain	√	enwik9	39.99	13.4596	70.85	PASSED
lastpos	\	enwik8	57.60	0.7498	127.20	PASSED
lastpos	√	enwik8	49.19	0.9543	99.94	PASSED
lastpos	\	enwik9	51.85	4.6463	205.26	PASSED
lastpos	√	enwik9	44.27	7.5464	126.37	PASSED

图 4: 引入 Deflate 压缩算法前后各指标对比

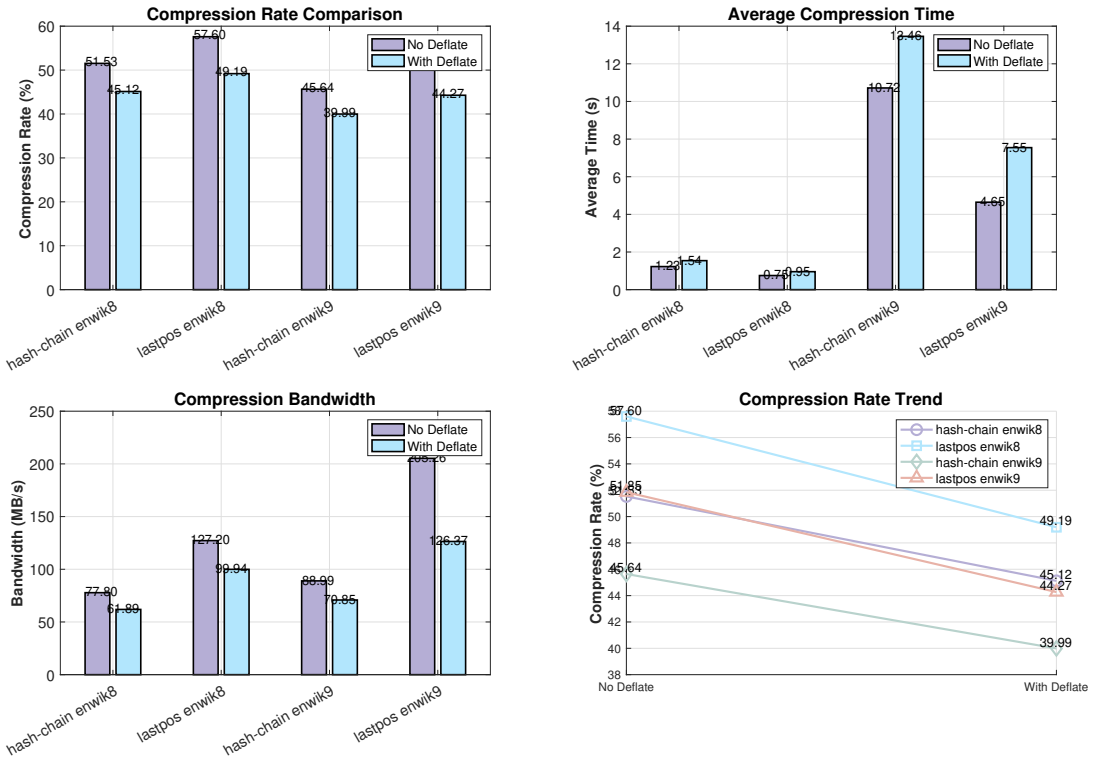

图 5: 引入 Deflate 压缩算法前后各指标对比

从实验结果可以看出，引入 Deflate 编码在所有测试场景下均能显著降低压缩率，但同时带来一定程度的压缩时间增长，从而导致压缩带宽下降。这一趋势在 Hash-Chain 与 Last-Pos 两种字典结构中表现出高度一致性，说明 Deflate 优化主要作用于编码阶段，而与前端匹配策略本身无关。

4.3.1 Hash-Chain 算法分析

在 Hash-Chain 算法中，引入 Deflate 后**压缩效果得到明显提升**。以 `enwik8` 为例，压缩率由 51.53% 降低至 45.12%；在 `enwik9` 上亦由 45.64% 下降至 39.99%。这表明 Huffman 编码能够有效压缩高频字面

量及短匹配所产生的输出冗余。

与此同时，**压缩耗时有所增加**，使得压缩带宽从 77.80 MB/s 降至 61.89 MB/s（`enwik8`），以及从 88.99 MB/s 降至 70.85 MB/s（`enwik9`）。该性能下降主要来源于 Deflate 编码阶段引入的比特级输出、长度码与距离码映射等额外计算开销。

4.3.2 Last-Pos 算法分析

对于 Last-Pos 算法，引入 Deflate 后的压缩率改善更加显著。实验结果显示，在 `enwik8` 数据集上，压缩率由 57.60% 降至 49.19%；在 `enwik9` 上由 51.85% 降至 44.27%。这说明 **Deflate 在一定程度上弥补了 Last-Pos 算法由于仅保留最近匹配位置而导致的匹配不足问题**。

从性能角度看，Last-Pos 在未引入 Deflate 时具有极高的压缩带宽（最高可达 205.26 MB/s），而引入 Deflate 后带宽下降至 126.37 MB/s。尽管如此，其压缩速度仍显著高于对应的 Hash-Chain + Deflate 实现，体现了 Last-Pos 在对性能敏感场景下的优势。

5 总结

本项目设计并实现了一套基于 LZ77 算法的高效文件压缩系统，通过逐步演进的优化策略，深入探究了数据结构对压缩算法“时间-空间”指标的影响。从基础的朴素匹配出发，引入了 Knuth 乘法哈希、前驱链表（Hash-Chain）、双路哈希（Dual-Hash Last-Pos）以及 Deflate 静态 Huffman 编码等技术进行逐步优化，最终构建了高性能的压缩工具。

5.1 核心工作总结

1. **哈希策略的有效性验证**：实验表明，Knuth 乘法哈希能有效打散数据分布，减少冲突。在此基础上，Hash-Chain 结构通过维护历史链表，实现了对历史匹配的深度挖掘，在压缩率上表现最优（enwik9 达到 39.99%）；而 Last-Pos 结构通过牺牲部分历史记忆换取了 $O(1)$ 的查找速度，其压缩带宽显著优于链表法。

2. **双路哈希（Dual-Hash）的应用**：针对 Last-Pos 算法“覆盖即遗忘”的缺陷，本项目提出的双路哈希策略（3 字节 +4 字节索引）是一个关键创新点。实验数据证明，该策略在不引入链表复杂度的前提下，有效挽回了大量丢失的匹配，使得 Last-Pos 算法在保持极高带宽（>120 MB/s）的同时，将压缩率控制在可用范围内。

3. **Deflate 熵编码的普适性提升**：引入 RFC 1951 标准的静态 Huffman 编码后，无论是 Hash-Chain 还是 Last-Pos，其压缩率均获得了 5% 至 8% 的显著提升。这一结果揭示了 LZ77（消除重复）与 Huffman（消除统计冗余）两种机制的互补性，证明了 Deflate 架构之所以成为工业界标准的合理性。

5.2 时空权衡与适用场景分析

本次实验的对比结果深刻揭示了算法设计中的权衡（Trade-off）规律：

- **存储敏感型场景**：推荐使用 **Hash-Chain + Deflate** 组合。虽然其压缩速度相对较慢（约 60-70 MB/s），但能提供最优的压缩率，适合用于归档存储、网络传输包压缩等对体积要求严格的场合。

- **实时性敏感型场景**：推荐使用 **Dual-Hash Last-Pos + Deflate** 组合。该方案在压缩率仅落后前者约 4% 的情况下，提供了近两倍的压缩速度（>120 MB/s），非常适合内存数据库快照、实时日志压缩或高吞吐量的云存储网关。

综上所述，本项目通过对 LZ77 及其变种算法的深入剖析与实现，为不同应用场景下的压缩算法选型提供了有力的数据支撑。